

Peer Review: Hur man granskar ett projekt-repo

Granskningsguide för förstaårsstudenter

INDA 25

2026-05-08

Varför gör vi Peer Review?

QA: Hitta buggar eller logiska luckor som författaren missat.

Varför gör vi Peer Review?

QA: Hitta buggar eller logiska luckor som författaren missat.

Kunskapsutbyte: Lära sig nya sätt att lösa problem och se hur andra strukturerar sin kod.

Varför gör vi Peer Review?

QA: Hitta buggar eller logiska luckor som författaren missat.

Kunskapsutbyte: Lära sig nya sätt att lösa problem och se hur andra strukturerar sin kod.

Professionalism: Träna på att ge och ta konstruktiv kritik – en av de viktigaste färdigheterna för en utvecklare.

1. Dokumentation & Setup

Innan du läser koden, titta på "utsidan":

README.md:

- Finns det en tydlig beskrivning av projektet?
- Är instruktionerna för att köra koden kompletta?

1. Dokumentation & Setup

Innan du läser koden, titta på "utsidan":

README.md:

- Finns det en tydlig beskrivning av projektet?
- Är instruktionerna för att köra koden kompletta?

Tänk att vem som helst ska kunna följa instruktionerna!

2. Kodkvalitet & Läsbarhet

Kod läses oftare än den skrivs. Granska följande:

- **Namngivning:** Är variabelnamn beskrivande? (Undvik `int x;`, föredra `int userCount;`).
- **Metodstorlek:** Är metoderna för långa? (En metod bör göra **en** sak).
- **Kommentarer:** Förklarar de **varför** något görs snarare än **vad** (koden ska tala för sig själv).

DRY (Don't Repeat Yourself):

- Ser du stora stycken kod som är kopierade? Tipsa om att bryta ut det till en gemensam metod.

3. Arkitektur & Design

Hur är logiken uppdelad?

- **Separation of Concerns:** Är UI-logik (print/input) separerad från affärslogiken?
- **Modularitet:** Är projektet uppdelat i logiska paket eller klasser?
- **Inkapsling:** Används `private` korrekt, eller är allt `public` utan anledning?

4. Tester & Robusthet

Enhetstester:

- Finns det tester för kärnfunktionaliteten?
- Täcker de "edge cases" (t.ex. tomma listor, null-värden, negativa tal)?

Felhantering:

- Kraschar programmet vid felaktig input?
- Används `try-catch` eller validering på ett rimligt sätt?

5. Git-historik & Repo-struktur

Ett snyggt repo handlar även om hur det har byggts.

- **Commit-meddelanden:** Är de informativa? ("fix bug" VS "Fix nullpointer in User login").
- **Små commits:** Har de checkat in stora block på en gång, eller är det en logisk historik?
- **Filstruktur:** Ligger .class-filer eller personliga inställningar (.idea/, .DS_Store) i repot? (Tipsa om .gitignore!).
- Används pull requests eller force-pushas allt?

Att ge feedback (Best Practices)

Gör	Undvik
Var specifik	Var vag ("Koden ser konstig ut")
Var konstruktiv	Var dömande ("Det här är dåligt")
Ställ frågor	Ge bara order ("Ändra detta till...")
Lyft fram det positiva	Fokusera enbart på felen

Exempel: "Jag ser att den här loopen körs två gånger, skulle man kunna kombinera dem för att öka prestandan?"

Sammanfattning: Checklisten

Gör ett issue för varje sak nedan:

- ☐ Branching and pull requests - Används branches och PRs på repot
- ☐ Kodkvalité - Är koden lättläst och välstrukturerad? Finns det tester
- ☐ Issues - används issues, finns labels osv
- ☐ Milestones and Features - Används milestones och med tags osv
- ☐ Commit log - commitas det ofta? har commits bra namn?
- ☐ Documentation & Instructions - finns det instruktioner, README och dokumentation. Är det lätt att följa de?

Kom ihåg: Målet är att hjälpa din kurskamrater att bli en bättre programmerare!